

Version 1.2

August 2026



MACHINE LEARNING

MODUL 6 - WEB MODEL DEPLOYMENT

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	1
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE OBJECTIVES.....	3
PREPARATION.....	3
KEYWORDS.....	3
MAIN TOPICS.....	4
1. Setup PDM.....	4
2. Key Concepts of Deployment.....	7
A. Data Preprocessing.....	7
B. Saving a Model.....	8
3. Streamlit.....	8
4. Creating a Streamlit Web App.....	10
5. Other Deployment Options: Flask & TensorFlow.js.....	16
A. Flask.....	17
1. What is Flask?.....	17
2. Advantages of Using Flask.....	17
3. Disadvantages of Using Flask.....	17
4. Documentation.....	17
B. TensorFlow.js.....	18
1. What is TensorFlow.js?.....	18
2. Advantages of Using TensorFlow.js.....	18
3. Disadvantages of Using TensorFlow.js.....	18
5. Documentation.....	18
CODELAB (10%).....	19
LAB ASSIGNMENT (90%).....	20



INTRODUCTION

OBJECTIVES

1. Students will understand the basic concepts of machine learning deployment models.
2. Students will understand the importance of data preprocessing and model storage before using them in applications.
3. Students will be able to use Streamlit to create machine learning web applications.
4. Students will understand how machine learning models, data preprocessing, and user interfaces integrate.
5. Students will be able to implement a simple deployment example for a real-world use case (e.g., a content-based movie recommender).

MODULE OBJECTIVES

1. Students will be able to set up PDM for managing project dependencies.
2. Students will be able to preprocess data and save it to ensure consistency during deployment.
3. Students will be able to save trained models in a file format so they can be used without retraining.
4. Students will be able to build an interactive web app with Streamlit to run machine learning models.
5. Students will understand the end-to-end workflow: data -> model -> deployment -> web app.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop/PC
3. Visual Studio Code or another IDE
4. Python (version ≥ 3.10 recommended)
5. PDM (Python Dependency Manager)
6. Streamlit
7. Supporting libraries: pandas, numpy, scikit-learn, matplotlib, joblib

KEYWORDS

Deployment, Machine Learning, Preprocessing, Model Saving, PDM, Streamlit, Web App, Python



MAIN TOPICS

With all the models we've built, we're used to getting prediction results directly through our code. Non-technical users may not be able to handle complex setups—from data preparation to generating predictions. To make these models accessible to non-technical users, we need to build an easy-to-use interface. One way to do that is to build a web application. To simplify building a web application for machine learning services, we'll use the Streamlit framework.

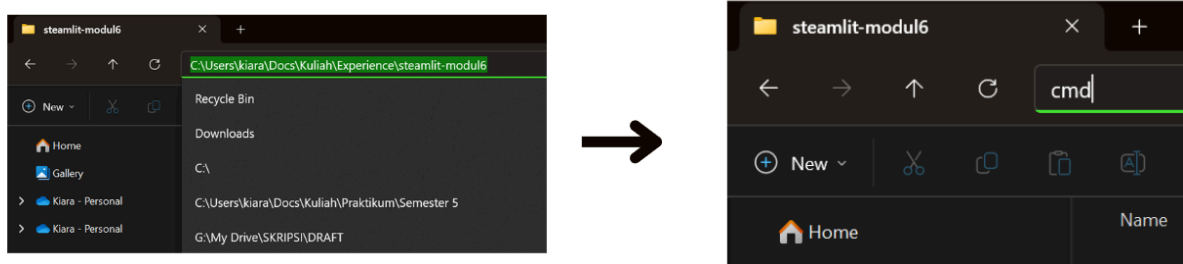
Streamlit is a Python-based framework focused on web-based data processing. It provides components we can use to make predictions on new data based on user input. To streamline application development, we'll also use PDM. PDM (**Python Dependency Manager**) is a package management tool. PDM is similar to venv, but it lets us choose the Python version, reduces version conflicts among installed libraries, and simplifies application development. However, before we build the web application, we must prepare the data preprocessing and model so we can use them later. Follow the steps below to build a machine learning website using Streamlit.

1. Setup PDM

First, make sure you have PDM installed on your laptop or PC. If not, open a terminal and enter the following command:

```
PS C:\Users\kiara> (Invoke-WebRequest -Uri https://pdm-project.org/install-pdm.py -UseBasicParsing).Content | python -i
```

Once it's installed, create a new folder for your project. Then navigate to that folder and open a terminal in the directory of your new folder:



Run the following command:

```
Microsoft Windows [Version 10.0.26100.3624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\kiara\Docs\Kuliah\Experience\steamlit-modul6>pdm init
```



If PDM isn't recognized globally, follow these steps:

1. Make sure PDM is installed
2. Check the versions of pip, Python, and PDM to ensure compatibility
3. Open a new terminal and try running the following command (pay attention to the path):

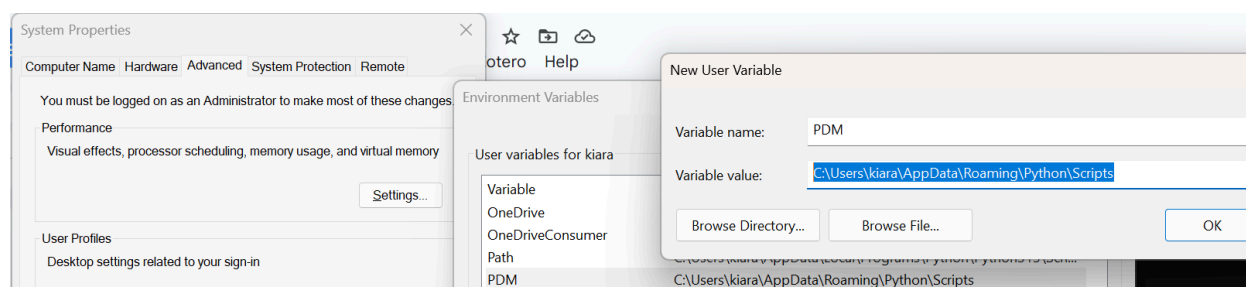
```
C:\Users\kiara>python -m pdm --version
PDM, version 2.25.7

C:\Users\kiara>python -m pdm init
Creating a pyproject.toml for PDM...
Please enter the Python interpreter to use
0. cpython@3.13 (C:\Users\kiara\AppData\Local\Programs\Python\Python313\python.EXE)
1. cpython@3.13 (C:\Users\kiara\.venv\Scripts\python.exe)
Please select (0): |
```

If both commands above run successfully, you're good to go, but you still need one more step to set the PDM variable so it's recognized globally. Add the user's **"Scripts"** folder to your PATH, or run the following command to find the path to your user's **"Scripts"** folder:

```
C:\Users\kiara>python -c "import site,os; print(os.path.join(site.USER_BASE,'Scripts'))"
C:\Users\kiara\AppData\Roaming\Python\Scripts
```

This will then produce an output like the one shown in the white box above. Copy that path and open it in Windows, then go to **Advanced System Settings -> Environment Variables -> User Variables -> New -> Paste the path you copied earlier.**



Once you're done, close and reopen your terminal. Navigate to the folder you're using for your project and run the command **`pdm init`**. If you see output similar to that in step 3, you've succeeded; if not, ask your assistant for help.



- After running the ``pdm init`` command, follow the instructions provided by pdm.

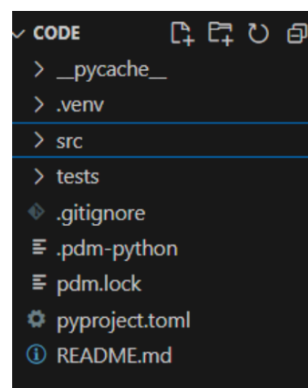
```
C:\Users\kiara\Docs\Kuliah\Experience\streamlit-modul6>pdm init
Creating a pyproject.toml for PDM...
INFO: Using the first matched interpreter.
Virtualenv is created successfully at C:\Users\kiara\Docs\Kuliah\Experience\streamlit-modul6\.venv
Project name (streamlit-modul6): streamlit-project
Project version (0.1.0): 0.1.0
Do you want to build this project for distribution(such as wheel)?
If yes, it will be installed by default when running `pdm install`. [y/n] (n): n
License(SPDX name) (MIT):
Author name (xyzaraa): xyzaraa
Author email (kiarazzahraaa@gmail.com):
Python requires('*' to allow any) (==3.13.*) : >=3.10
INFO: Git repository initialized successfully.
Project is initialized successfully
```

- **Project name:** your project name
- **Project version:** This isn't the Python version, but the version of your own project. The default is 0.1.0 (SemVer standard: major.minor.patch)
- **License:** You can fill this in, but if you don't, just press Enter to skip it
- **Author and email:** Optional; if left blank, just press Enter
- **Python requirements:** The Python version you're using for the project. For example, if you're using Python 3.13, you can write `>=3.10`, or if you're certain you're only using one specific version, you can write `==3.13.0`. However, if any Python version is acceptable, enter `*`.

- Next, add any other required dependencies. Wait until the installation is complete.

```
C:\Users\kiara\Docs\Kuliah\Experience\streamlit-modul6>pdm add streamlit==1.37.0 tensorflow scikit-learn joblib
Adding packages to default dependencies: streamlit==1.37.0, tensorflow, scikit-learn, joblib
```

- By default, PDM will automatically create several files and folders. We'll use the ``src`` folder to store the code we'll be writing. There's also a file named ``pyproject.toml`` that lists the libraries we've installed in the project, and we can add scripts to make it easier to run our application.



2. Key Concepts of Deployment

A. Data Preprocessing

During training, we often transform the data first before feeding it into the model. For example:

- Tabular data: We use **StandardScaler** to standardize the data so that all features are on the same scale.
- Text data: We use a Tokenizer or **TF-IDF Vectorizer** to convert the text into numerical values.
- Image data: We resize the images to a specific size or normalize the pixels to a scale of 0–1.

The problem is that if new data becomes available later, you must preprocess it the same way you did during training. Otherwise, the model's predictions may be incorrect. Therefore, you must save the preprocessing to a file (for example, a **.pkl file using joblib**). This is extremely important because machine learning is highly sensitive to input data format. If the data used for predictions in Streamlit isn't processed the same way as the data used during training, accuracy will decrease.

Sample code for saving scalers when training a model using the ``dump`` function from the joblib library:

```
from sklearn.preprocessing import StandardScaler
import joblib

scaler = StandardScaler().fit(X_train)
joblib.dump(scaler, "src/model/scaler.pkl")
```

If you want to load the scaler when running the application, use the following code example with the ``load`` function from the ``joblib`` library:

```
import joblib
scaler = joblib.load("src/model/scaler.pkl")
X_new_scaled = scaler.transform(X_new)
```

This ensures new data is processed the same way as the training data.



B. Saving a Model

Once a model is trained, you don't need to retrain it every time the application runs. Train it once, then save it to a file. If you train the model with scikit-learn, use joblib to save it as a .pkl file. However, if you train the model using Keras/TensorFlow, use the Keras function **model.save()** directly.

3. Streamlit

Streamlit is a Python framework for building interactive web apps, particularly for data science, machine learning, and visualization. Benefits of Streamlit include not needing HTML, CSS, or JavaScript—you only need Python—and being able to view the app by running ``streamlit run app.py``. Additionally, Streamlit integrates directly with Python libraries such as pandas, numpy, matplotlib, plotly, scikit-learn, and others, making it ideal for dashboards, ML prototyping, and data applications.

To install Streamlit, simply use pip with the following command:

```
pip install streamlit
```

Alternatively, you can also add it using the PDM you've already initialized:

```
pdm add streamlit
```

There are several functions you can use to create a web app with Streamlit:

Component	STREAMLIT	
	Code	Functions
Title	st.title()	Creating a large heading on a page
Header	st.header()	Creating a header section
Subheader	st.subheader()	Creating a subheader
Write	st.write()	Displaying text, numbers, dataframes, or plots
Markdown	st.markdown()	Displaying text in Markdown format
Button	st.button()	Creating a clickable button
Checkbox	st.checkbox()	Creating a checkbox



Radio	<code>st.radio()</code>	Single-choice (radio button)
Selectbox	<code>st.selectbox()</code>	Dropdown to select one item
Multiselect	<code>st.multiselect()</code>	Dropdown for selecting multiple items
Slider	<code>st.slider()</code>	Enter a number using the slider
Text Input	<code>st.text_input()</code>	Text input
Number Input	<code>st.number_input()</code>	Number input
File Uploader	<code>st.file_uploader()</code>	Upload a file (e.g., CSV)
Columns	<code>col1, col2 = st.columns()</code> with <code>col1: st.write()</code> with <code>col2: st.write()</code>	Splitting a page into multiple columns
Tabs	<code>tab1, tab2 = st.tabs(["Tab 1", "Tab 2"])</code> with <code>tab1: st.write("Isi tab 1")</code>	Creating navigation tabs on a page
Sidebar	<code>st.sidebar.title("Menu")</code> <code>choice = st.sidebar.selectbox("Pilih:", ["Home", "About"])</code>	Creating a menu in the sidebar
Expander	with <code>st.expander("Lihat detail"):</code> <code>st.write("Isi tersembunyi")</code>	Creating sections that can be expanded or collapsed
Container	with <code>st.container():</code> <code>st.write("Konten")</code>	Create a custom container (you can add a border style)
Divider	<code>st.divider()</code>	Creating a horizontal separator line
DataFrame	<code>st.dataframe(df)</code>	Displaying an interactive dataframe
Table	<code>st.table(df)</code>	Displaying a static (non-interactive) table
Metric	<code>st.metric(label="Accuracy", value="95%", delta="+2%")</code>	Display key figures with delta (+/-)
Line Chart	<code>st.line_chart(df)</code>	Creating a Simple Line Chart
Bar Chart	<code>st.bar_chart(df)</code>	Creating a simple bar chart
Area Chart	<code>st.area_chart(df)</code>	Creating a simple area chart



Matplotlib	st.pyplot(fig)	Displaying a plot from Matplotlib
Plotly	st.plotly_chart(fig)	Displaying a plot from Plotly
Altair	st.altair_chart(fig)	Displaying the plot from Altair
Bokeh	st.bokeh_chart(fig)	Showing the Bokeh plot
Image	st.image("gambar.png", width=300)	Displaying an image
Audio	st.audio("suara.mp3")	Display the audio player
Video	st.video("video.mp4")	Display the video player

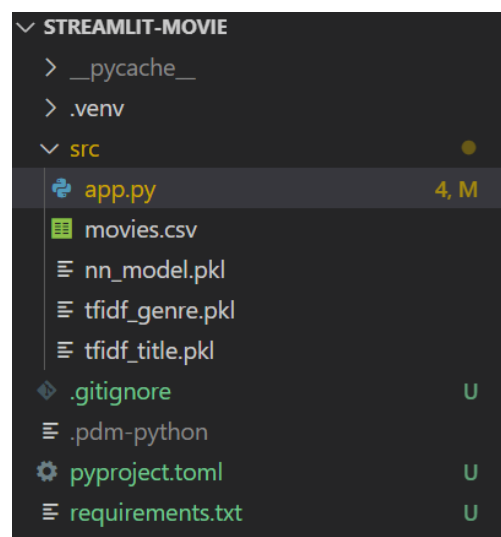
4. Creating a Streamlit Web App

To learn more, let's build an app using Streamlit. To access the sample code below, click the following link: [MODEL MODULE 6](#). Be sure to download all the files in that folder.

movieId		title	genres	userId	rating	timestamp
74299	7254	The Butterfly Effect (2004)	Drama Sci-Fi Thriller	608	4.0	1117162467
9992	355	Flintstones, The (1994)	Children Comedy Fantasy	222	2.0	1391351910
96109	104944	Short Term 12 (2013)	Drama	610	4.0	1479542945
32564	1544	Lost World: Jurassic Park, The (1997)	Action Adventure Sci-Fi Thriller	71	3.0	864738453

Sample Dataset Movie Recommender

Make sure you already have a folder set up for this project. Organize the file and folder structure neatly to make it easier for the program to access the paths to your model files.



OK, let's get to the content. First, set up the libraries needed to create a dashboard using Streamlit. Streamlit creates interactive dashboards, pandas reads and processes the dataset, and joblib loads the pre-trained TF-IDF model. The re module helps with text manipulation using regular expressions. From scikit-learn, we import `TfidfVectorizer` to extract text features and `NearestNeighbors` to calculate the similarity between movies. Additionally, we import `hstack` from `scipy.sparse` to combine sparse matrices, as well as `get_close_matches` from `difflib` to provide movie title suggestions in case of typos.

```
import streamlit as st
import pandas as pd
import joblib
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack
from sklearn.neighbors import NearestNeighbors
from difflib import get_close_matches
```

Import the required libraries.

Next, set the browser tab title (not the page title). The goal is that when you open the app, the browser tab will display **"Content-Based Movie Recommendation."** The `page_title` parameter sets the tab title, while `layout="wide"` makes the app display in full-width mode, giving you more flexibility when displaying components. You can rename this option as you like.

```
st.set_page_config(page_title="Content-Based Movie Recommendation", layout="wide")
```

Next, we'll create a helper function, `remove_year_from_title`, which removes the year typically included at the end of a movie title—for example, **"Toy Story (1995)"** becomes **"Toy Story."** Meanwhile, the `to_norm` function normalizes the string by converting all letters to lowercase and removing excess spaces. This ensures title searches aren't case- or space-sensitive.

```
# =====
# Helpers
# =====
def remove_year_from_title(title: str) -> str:
    return re.sub(r"(\d{4})", "", str(title)).strip()

def to_norm(s: str) -> str:
    return re.sub(r"\s+", " ", s.lower()).strip()
```



Next, we need data-loading and model-loading functions as the core of the initialization process. First, the application loads **movies.csv** and **ratings.csv**, then merges them using `movieId`. We remove columns not required for content-based filtering—such as `timestamp`, `userId`, and `rating`—but intentionally retain `movieId` because it serves as the synchronization anchor between training and serving. To ensure each row represents a single movie, we deduplicate the data by `movieId`. Next, we preprocess the data identically to the model training stage (our model removes the year from the title, splits the genre into a list of tokens, removes the original `genres` column, and creates a `title_norm` column from the normalized title). Once the data is organized, the saved artifacts (files from the training process) are loaded with `joblib.load`, so the web app doesn't need to retrain the model every time it runs.

Next, the serving-side row order is aligned with the training order using `titles_index.csv` (which contains `training_index` and `movieId`). We merge this file into the dataframe on `movieId`, then sort by `training_index` to ensure the indices returned by `NearestNeighbors` always point to the correct movie (avoiding duplicate titles or incorrect recommendations).

After the sequences are synchronized, we reconstruct the feature matrix `X`: the `title` column is transformed using `tfidf_title`, the `genre` column (which has already been merged by row) is transformed using `tfidf_genre`, and then both are combined using `hstack`. We add a simple sanity check—comparing the number of rows in `X` with the number of rows the model recognized during training—to ensure that the artifacts and data truly match. Finally, we create a `norm_to_index` map that maps `title_norm` to row indices, so that case-insensitive title searches are fast and accurate. The function then returns `df`, `X`, `nn`, and `norm_to_index`—all four of which the recommendation function and the Streamlit UI use without retraining the model.



```
# Load data & load model
# =====
@st.cache_resource(show_spinner=True)
def load_data_and_model():
    movies = pd.read_csv("movies.csv")
    ratings = pd.read_csv("ratings.csv")

    df = pd.merge(movies, ratings, on="movieId")
    # buang kolom tidak dipakai, TAPI simpan movieId
    df = df.drop(columns=["timestamp", "userId", "rating"], errors="ignore")

    # Satu baris per film biar konsisten
    df.drop_duplicates(subset=["movieId"], inplace=True)

    #Preprocessing sama seperti training
    df["title"] = df["title"].apply(remove_year_from_title)
    df["genre"] = df["genres"].apply(lambda x: str(x).split("|"))
    df = df.drop(columns=["genres"], errors="ignore")
    df["title_norm"] = df["title"].apply(to_norm)

    tfidf_title: TfidfVectorizer = joblib.load("tfidf_title.pkl")
    tfidf_genre: TfidfVectorizer = joblib.load("tfidf_genre.pkl")
    nn: NearestNeighbors = joblib.load("nn_model.pkl")
    idx_map = pd.read_csv("titles_index.csv") # cols: training_index, movieId, title
    idx_map = idx_map.drop_duplicates(subset=["movieId"]).sort_values("training_index")

    # re-order df agar PERSIS seperti saat training
    df = idx_map[["movieId"]].merge(df, on="movieId", how="left", sort=False)

    X_title = tfidf_title.transform(df["title"])
    X_genre = tfidf_genre.transform(df["genre"].apply(lambda xs: " ".join(xs)))
    X = hstack([X_title, X_genre])

    # sanity check
    try:
        n_fit = getattr(nn, "_fit_X", None).shape[0]
        if n_fit != X.shape[0]:
            st.error(
                f"Model/data mismatch: model rows={n_fit}, current rows={X.shape[0]}. "
                "Pastikan titles_index.csv dibuat dari df yang sama saat fit."
            )
    except Exception:
        pass

    # mapping judul_norm -> index
    norm_to_index = {t: i for i, t in enumerate(df["title_norm"].values)}

    return df, X, nn, norm_to_index

df, X, nn, norm_to_index = load_data_and_model()
```

For the final “**backend**” touch, we’ll create a recommendation function. The `recommend_movies` function accepts a movie title and the number of recommendations. We normalize the input with `to_norm` and then check it against `norm_to_index`. If the title isn’t found, the system collects `title_norm` candidates and suggests similar titles using `get_close_matches` to ensure a seamless experience. If the title is found, the function retrieves the movie’s index, calls `nn.kneighbors` on the relevant row vectors in `X`, and then iterates through the nearest neighbors while avoiding duplicates: the row itself and titles with the same



name. The scores are converted to **similarity = 1 - distance**, so a higher value means the titles are more similar. If no suitable candidates remain after filtering, the function returns a message stating that no recommendations were found.

```
def recommend_movies(movie_title: str, num_recommendations: int = 5):
    query_norm = to_norm(movie_title)

    if query_norm not in norm_to_index:
        # saran judul mirip
        candidates = list(norm_to_index.keys())
        suggestions = get_close_matches(query_norm, candidates, n=5, cutoff=0.6)
        return [], (
            "Film tidak ditemukan. Mungkin maksud Anda: "
            + ", ".join([df["title"].iloc[norm_to_index[s]] for s in suggestions])
            if suggestions else "Film tidak ditemukan dalam dataset."
        )

    idx = norm_to_index[query_norm]
    k = min(num_recommendations + 50, X.shape[0])
    distances, indices = nn.kneighbors(X[idx], n_neighbors=k)

    recs, seen_idx, seen_titles = [], {idx}, {df["title"].iloc[idx]}
    for dist, i in zip(distances[0], indices[0]):
        title_i = df["title"].iloc[i]
        # dedup: skip diri sendiri & judul yang sama
        if i in seen_idx or title_i in seen_titles:
            continue
        similarity = 1.0 - float(dist) # cosine similarity
        recs.append((title_i, similarity))
        seen_idx.add(i)
        seen_titles.add(title_i)
        if len(recs) >= num_recommendations:
            break

    if not recs:
        return [], "Tidak ada rekomendasi yang ditemukan."
    return recs, ""
```

Finally, in terms of the UI, the web app will display the title at the top and a summary of the number of available movies. For context, the full list of titles is in an expander so users can open it when needed without distracting from the main focus. The user types a title into `st.text_input` and specifies the number of recommendations via `st.number_input`. When input is available, the app calls the recommendation function and displays the results as a list of titles along with their similarity scores, rounded to three decimal places. If no title is found, it displays a warning and suggests the closest match; however, if no candidates are sufficiently similar, it displays a message stating that no recommendations are available. This workflow ensures the



saved model is used as-is and the data order remains consistent, resulting in stable, reasonable recommendations.

```
st.title("Content-Based Movie Recommendation")

st.subheader(f"Daftar Film Tersedia ({df['title'].unique()})")
with st.expander("Lihat semua judul"):
    st.caption("Daftar ini hanya untuk referensi – ketik judul di kotak pencarian di bawah.")
    st.dataframe(
        pd.DataFrame({"Title": sorted(df["title"].unique())}),
        use_container_width=True,
        hide_index=True
    )

movie_input = st.text_input("Judul film:")
num_recommendations = st.number_input("Jumlah rekomendasi:", min_value=1, max_value=20, value=5, step=1)

if movie_input:
    recs, msg = recommend_movies(movie_input, num_recommendations)
    if msg and "Film tidak ditemukan" in msg:
        st.warning(msg)
    elif recs:
        st.subheader(f"Rekomendasi untuk '{remove_year_from_title(movie_input)}':")
        for i, (title, sim) in enumerate(recs, start=1):
            st.write(f"{i}. {title} – Similarity: {sim:.3f}")
    if msg and "Tidak ada rekomendasi" in msg:
        st.info(msg)
```

After writing all the code above (make sure to type it out yourself, guys, so you understand the flow—even if you're just copy-pasting 😊), run the following command in your terminal, making sure the path matches the location of your .py file

```
(streamlit-movie-3.13) PS C:\Users\kiara\Docs\Kuliah\Experience\streamlit-movie> cd src
(streamlit-movie-3.13) PS C:\Users\kiara\Docs\Kuliah\Experience\streamlit-movie\src> streamlit run app.py

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8503
Network URL: http://192.168.110.57:8503
```

Once **"localhost"** appears, the program will automatically redirect you to your web app; if it doesn't, just click the local URL.



The result of the code we wrote above will appear as follows:

Content-Based Movie Recommendation

Daftar Film Tersedia (9446)

Lihat semua judul

Daftar ini hanya untuk referensi — ketik judul di kotak pencarian di bawah.

	Title
0	'71
1	'Hellboy': The Seeds of Creation
2	'Round Midnight
3	'Salem's Lot
4	'Til There Was You
5	'Tis the Season for Love
6	'burbs, The
7	'night Mother
8	(500) Days of Summer
9	*batteries not included

List of Movies Available in the Dataset

Content-Based Movie Recommendation

Daftar Film Tersedia (9446)

Lihat semua judul

Judul film:

paranormal activity

Jumlah rekomendasi:

7

Rekomendasi untuk 'paranormal activity':

1. Paranormal Activity 3 — Similarity: 0.890
2. Paranormal Activity 2 — Similarity: 0.712
3. Paranormal Activity 4 — Similarity: 0.712
4. You're Next — Similarity: 0.707
5. They — Similarity: 0.707
6. As Above, So Below — Similarity: 0.707
7. M/M/S/G — Similarity: 0.707

Showing the Nearest Recommendations

5. Other Deployment Options: Flask & TensorFlow.js

Streamlit is great for quickly building a dashboard, but it's not the only way to deploy a machine learning model on the web. Two other common options are **Flask** and **TensorFlow.js**. Both take a very different approach from Streamlit, so it's worth understanding what they are and when they make sense to use.



A. Flask

1. What is Flask?

Flask is a lightweight Python web framework for building web applications and APIs. Unlike Streamlit – which automatically generates the UI for you based on simple function calls like `st.button()` or `st.slider()` – Flask gives you full control over both the backend logic and the frontend design (HTML, CSS, JavaScript). You can also use Flask to expose a model purely as an API, with no visible webpage, so other applications (mobile apps, JavaScript frontends, other servers) can send data and get predictions back.

In short: Streamlit builds the interface *for* you, while Flask lets you build *your own* interface – or skip the interface entirely and just serve predictions as data.

2. Advantages of Using Flask

- **Full control** over how the app looks and behaves – not limited to Streamlit's pre-built components.
- You can build a **REST API**, making the model usable by other apps, not just a single web dashboard.
- Better suited for **production-grade** applications – supports authentication, databases, custom routing, etc.
- Lightweight and flexible – you only add what you need.

3. Disadvantages of Using Flask

- Requires you to write your own frontend (HTML/CSS/JS) if you want a visible UI – more work than Streamlit.
- No built-in interactive widgets; you must build everything (forms, buttons, charts) manually.
- Steeper learning curve for students who are only used to plain Python scripts, since it introduces web development concepts (routes, requests, templates).
- Slower to prototype with compared to Streamlit.

4. Documentation

Flask official documentation: [FLASK DOCUMENTATION](#)



B. TensorFlow.js

1. What is TensorFlow.js?

TensorFlow.js (TF.js) is a JavaScript library that lets machine learning models run directly in the browser, on the user's device—not on a server. This means once the browser loads the model, predictions happen locally using the user's CPU/GPU, without sending data back and forth to a backend.

This differs fundamentally from Streamlit and Flask, where the model runs on the server, and the browser just displays results. With TensorFlow.js, the "server" role mostly disappears at prediction time — the model itself is shipped to the client.

2. Advantages of Using TensorFlow.js

- **Runs entirely in the browser** — no need for a Python backend server to make predictions.
- **Low latency**, since there's no network round-trip for each prediction.
- Better for **privacy-sensitive** applications, since user data (e.g., images, text) never has to leave their device.
- Can work **offline** once the model and page are loaded.
- Useful for browser-based interactive experiences (e.g., webcam-based image classification, real-time pose detection).

3. Disadvantages of Using TensorFlow.js

- Model files download to the browser, so large models load slowly and use a lot of bandwidth and memory— not ideal for big deep learning models.
- You must first convert your trained Keras/TensorFlow model into a TF.js-compatible format.
- Development is done in **JavaScript**, not Python, which may be unfamiliar for students coming from a Python-only ML background.
- Debugging and performance tuning happen in the browser environment, which behaves differently from a Python server.

5. Documentation

TensorFlow.js official documentation: [TENSORFLOW.JS DOCUMENTATION](https://www.tensorflow.org/js)

It's really easy, isn't it? 😊 Since it's so easy, go ahead and complete the Codelab and lab exercises below 🤖🎉



CODELAB (10%)

Follow the instructions in the material above, from setting up PDM to building a dashboard using Streamlit. If you have any questions, please ask the assistant.

DETAILS OF CODELAB'S WORK

DETAIL	POINT
Completing the PDM Setup	50
Successfully Built a Dashboard Based on the Material	50
TOTAL	100



LAB ASSIGNMENT (90%)

Create a dashboard that incorporates all the models (**image classification, tabular, and text**) you created in the previous module (Module 5). Make the dashboard as engaging as possible, and explain to the teaching assistant how you created it and how it works. For those who want a perfect grade (100%), you can experiment with the other model and dataset (In addition to Module 5), and then you can transform it into a web dashboard (Flask or TensorFlow.js).

FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment	Design Dashboard	15%
	Result Deployment	15%
	Extra Model	15%
Answering Question		45%
Total		100%

